

Applied Cyber Threat Analysis – YARA Lab

- This lab was conducted within the Virginia Cyber Range Environment.
- Report based off assignment assigned by George Mason University - Department of Information Sciences and Technology College of Engineering and Computing
- Cheche Agada. (2024). Email Header Analysis Lab IT-462: Applied Cyber Threat Analysis, George Mason University

Introduction:

Yet Another Recursive Acronym (YARA) is an open-source pattern-matching tool, used for detecting and classifying malicious threats. It uses a set of instructions called rules. A YARA rule defines the characteristics of a specific type of malicious element or activity. The rules are run against suspicious files or programs to detect malicious activity. In this lab, we will demonstrate the construction of YARA rules and their application in the detection of malicious activity.

We identify a suspicious file in our network to investigate/analyze it to determine that it is a malicious program. We use it to create a YARA rule, then we use the rule to scan other files and folders on the network in search of the identified malicious file.

Objective:

Demonstrate YARA rule writing and its application in malware detection.

Step 1 – Download YARA on Windows

- Download and unzip YARA (<https://github.com/VirusTotal/yara/releases/tag/v4.2.3>)

YARA v4.2.3 Latest

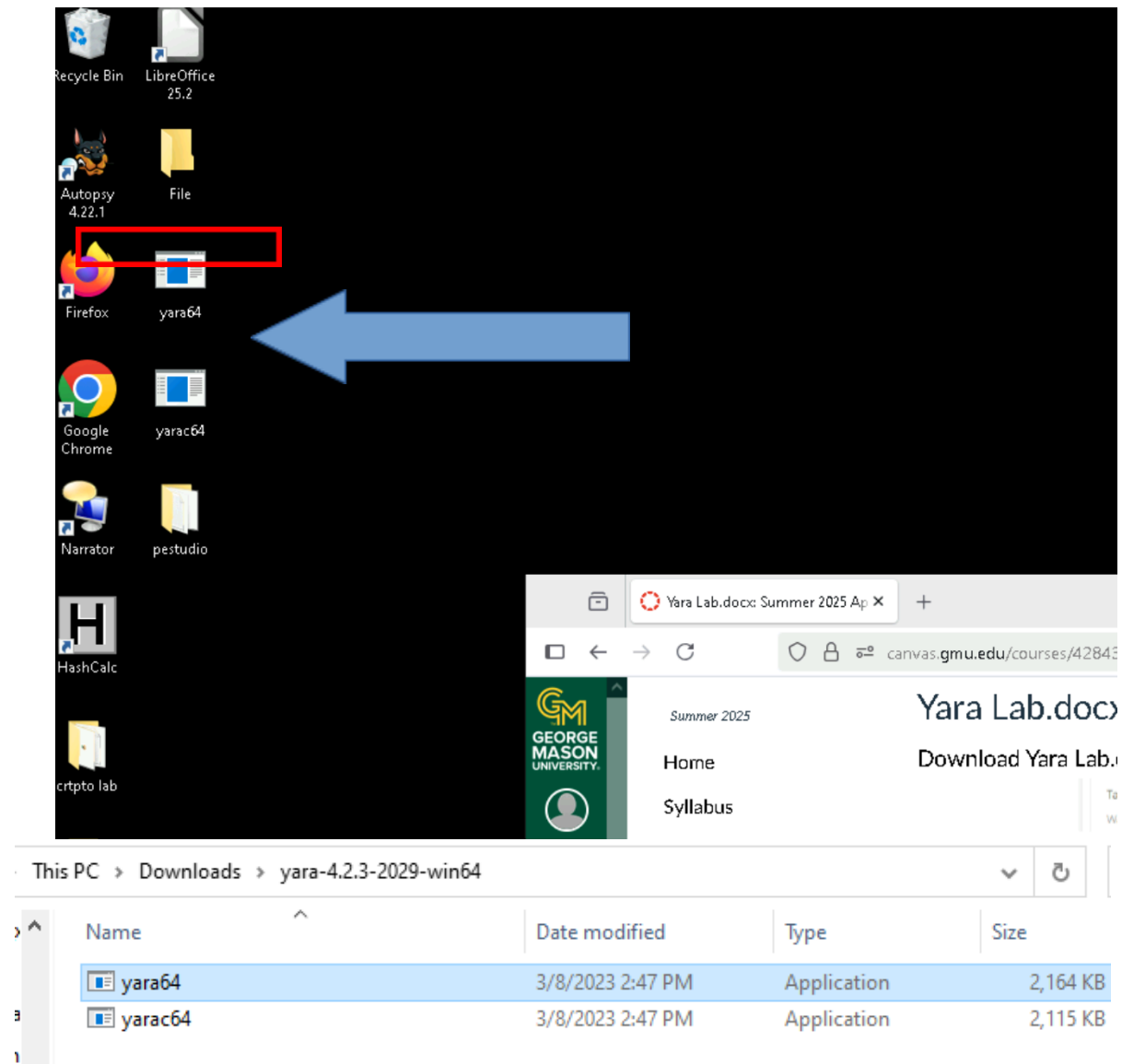
plusvic released this Aug 9, 2022 v4.2.3 ba94b4f

- BUGFIX: Fix security issue that can lead to arbitrary code execution ([b77e4f4](#) , [b77e4f4](#)). Thanks to ANSSI - CERT-FR for the report.
- BUGFIX: Fix incorrect logic in expressions like `<quantifier>` of `<string_set>` in `(start..end` ([#1757](#)).

Assets 4

yara-4.2.3-2029-win32.zip	1.37 MB	Aug 8, 2022
yara-4.2.3-2029-win64.zip	1.99 MB	Aug 8, 2022
Source code (zip)		Aug 8, 2022
Source code (tar.gz)		Aug 8, 2022

👍 17 🔗 1 18 people reacted



Step 2 – Download PeStudio

Portable Executables (PEs) are executable files in Windows systems. A binary in this case is another name for a Windows executable file. PeStudio is an analysis tool that inspects binaries without executing them. With PeStudio, we can identify file information such as hashes, size, type, tooling, etc.

- Download and unzip PeStudio (<https://pestudio.en.lo4d.com/download>)

Troy Fracyon
6/15/2025

https://pestudio.en.lo4d.com/download

LO4D.com

Software Utilities Computer Utilities **PeStudio**

 **PeStudio**
9.48 Portable executable analysis app for Windows

Advertisement

 **Tap "Download" to Start**
Tap "Download" to Start
Wave Browser

Mirrors for PeStudio:

-  [LO4D.com UK](#)
-  [LO4D.com US](#)
-  [US Mirror #1](#)

This PC > Downloads > pestudio > pestudio

Name	Date modified	Type	Size
xml	3/8/2023 3:08 PM	File folder	
changes	3/8/2023 3:08 PM	Text Document	18 KB
peparser.dll	3/8/2023 3:08 PM	Application exten...	1,346 KB
 pestudio	3/8/2023 3:08 PM	Application	849 KB

cycle Bin LibreOffice 25.2

Autopsy 4.22.1 File

Firefox yara64

Google Chrome yara64

Narrator pestudio

dashCalc

Yara Lab.docx: Summer 2025 Ap X

canvas.gmu.edu/cc

Summer 2025

Home

Yara La

Download

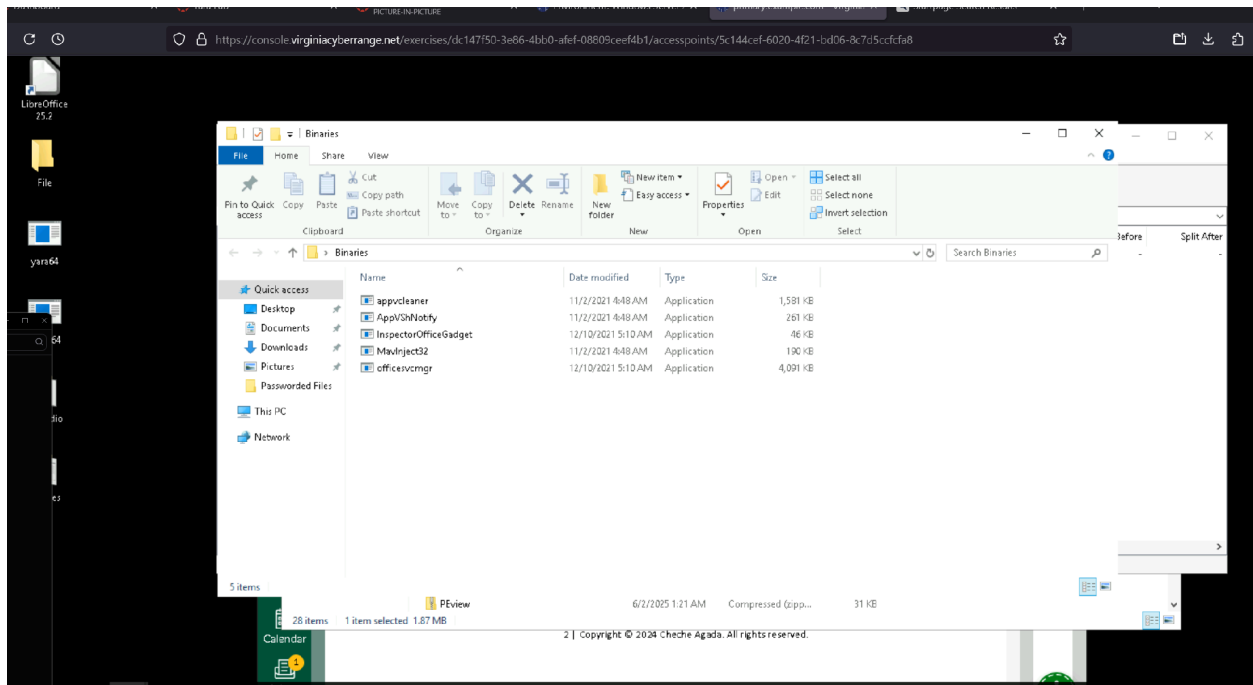


Troy Fracyon
6/15/2025

Step 3 – Create a YARA rule

- A sample 'Binaries' folder (**Binaries.rar**) was provided for this lab. Download and unzip.

Name	Status	Date modified	Type	Size
appvcleaner	✓	11/2/2021 12:48 AM	Application	1,581 KB
AppVShNotify	✓	11/2/2021 12:48 AM	Application	261 KB
InspectorOfficeGadget	✓	12/10/2021 12:10 AM	Application	46 KB
MavInject32	✓	11/2/2021 12:48 AM	Application	190 KB
officesvcmgr	✓	12/10/2021 12:10 AM	Application	4,091 KB



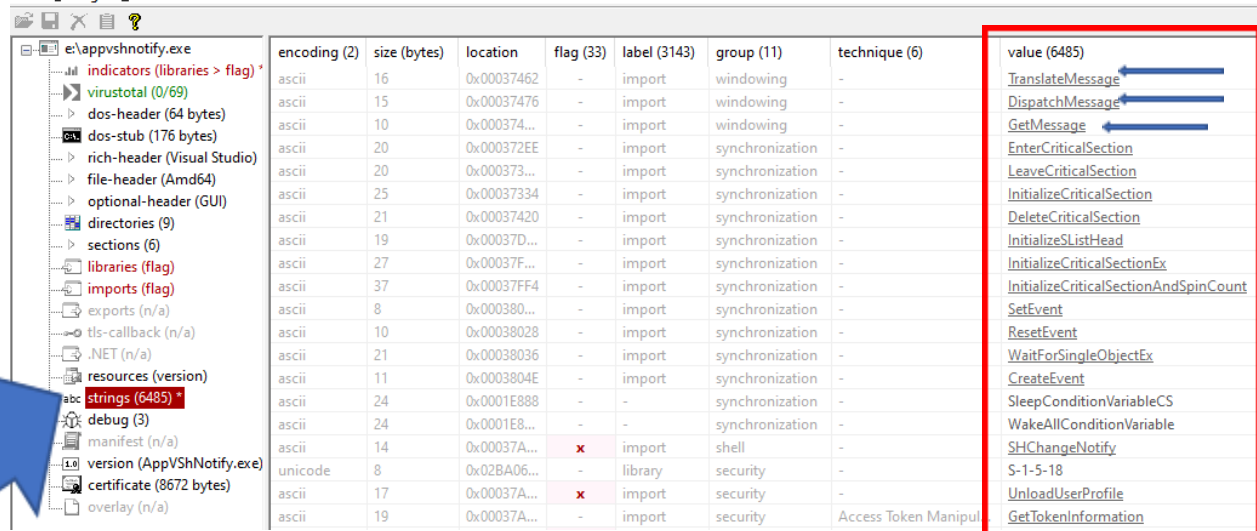
- Select a binary (i.e., AppVShNotify.exe) and open it in PeStudio

Troy Fracyon

6/15/2025

pestudio 9.48 - Malware Initial Assessment - www.winitor.com - [e:\appvshnotify.exe]

file settings about



encoding (2)	size (bytes)	location	flag (33)	label (3143)	group (11)	technique (6)	value (6485)
ascii	16	0x00037462	-	import	windowing	-	TranslateMessage
ascii	15	0x00037476	-	import	windowing	-	DispatchMessage
ascii	10	0x000374...	-	import	windowing	-	GetMessage
ascii	20	0x000372EE	-	import	synchronization	-	EnterCriticalSection
ascii	20	0x000373...	-	import	synchronization	-	LeaveCriticalSection
ascii	25	0x00037334	-	import	synchronization	-	InitializeCriticalSection
ascii	21	0x00037420	-	import	synchronization	-	DeleteCriticalSection
ascii	19	0x00037D...	-	import	synchronization	-	InitializeListHead
ascii	27	0x00037F...	-	import	synchronization	-	InitializeCriticalSectionEx
ascii	37	0x00037FF4	-	import	synchronization	-	InitializeCriticalSectionAndSpinCount
ascii	8	0x000380...	-	import	synchronization	-	SetEvent
ascii	10	0x00038028	-	import	synchronization	-	ResetEvent
ascii	21	0x00038036	-	import	synchronization	-	WaitForSingleObjectEx
ascii	11	0x0003804E	-	import	synchronization	-	CreateEvent
ascii	24	0x0001E888	-	-	synchronization	-	SleepConditionVariableCS
ascii	24	0x0001E8...	-	-	synchronization	-	WakeAllConditionVariable
ascii	14	0x00037A...	x	import	shell	-	SHChangeNotify
unicode	8	0x02BA06...	-	library	security	-	S-1-5-18
ascii	17	0x00037A...	x	import	security	-	UnloadUserProfile
ascii	19	0x00037A...	-	import	security	Access Token Manipul	GetTokenInformation

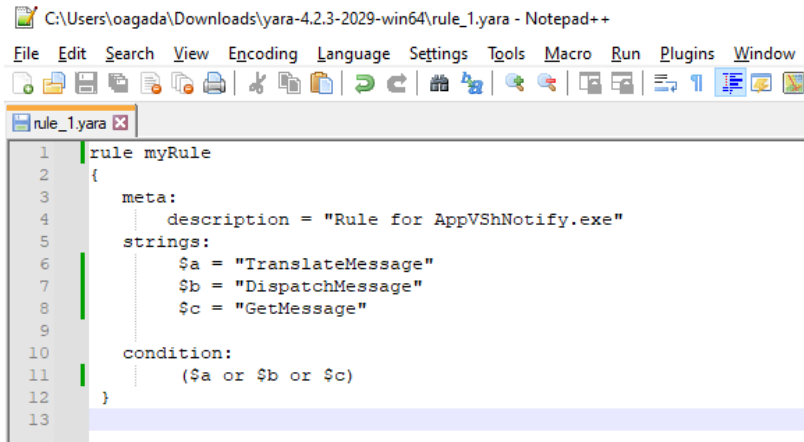
- Select 'strings' on the left pane in PeStudio
- Copy out a few strings from the 'value' pane to the right (i.e., TranslateMessage, DispatchMessage, GetMessage, etc.)
- Use the copied strings to write Yara rules as described below
- Copy and paste the following rule into a text file (i.e. Notepad++)
- Save the file (i.e., 'rule_1.yara')

YARA RULE

```
rule myRule
# declares rule and names it myRule
{
  meta:
    description = "Rule for AppVShNotify.exe"
  # target of analysis is AppVShNotify.exe
  strings:
    $a = "TranslateMessage"
    $b = "DispatchMessage"
    $c = "GetMessage"
  # $ defines search terms that YARA will look for in file

  condition:
    ($a or $b or $c)
}
```

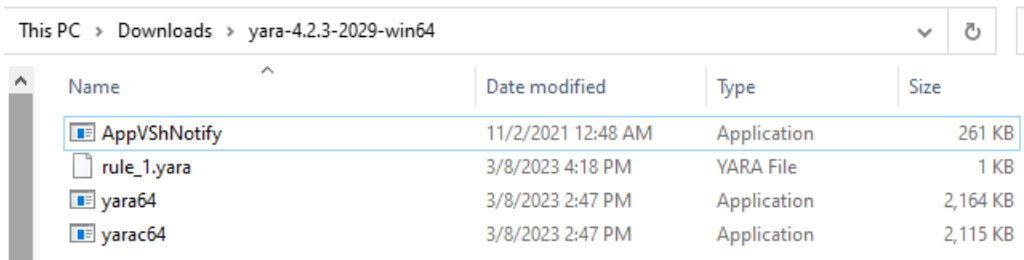
logic rule that uses "or" condition to determine if rule needs to be triggered



```
1 rule myRule
2 {
3     meta:
4         description = "Rule for AppVShNotify.exe"
5     strings:
6         $a = "TranslateMessage"
7         $b = "DispatchMessage"
8         $c = "GetMessage"
9
10    condition:
11        ($a or $b or $c)
12 }
13
```

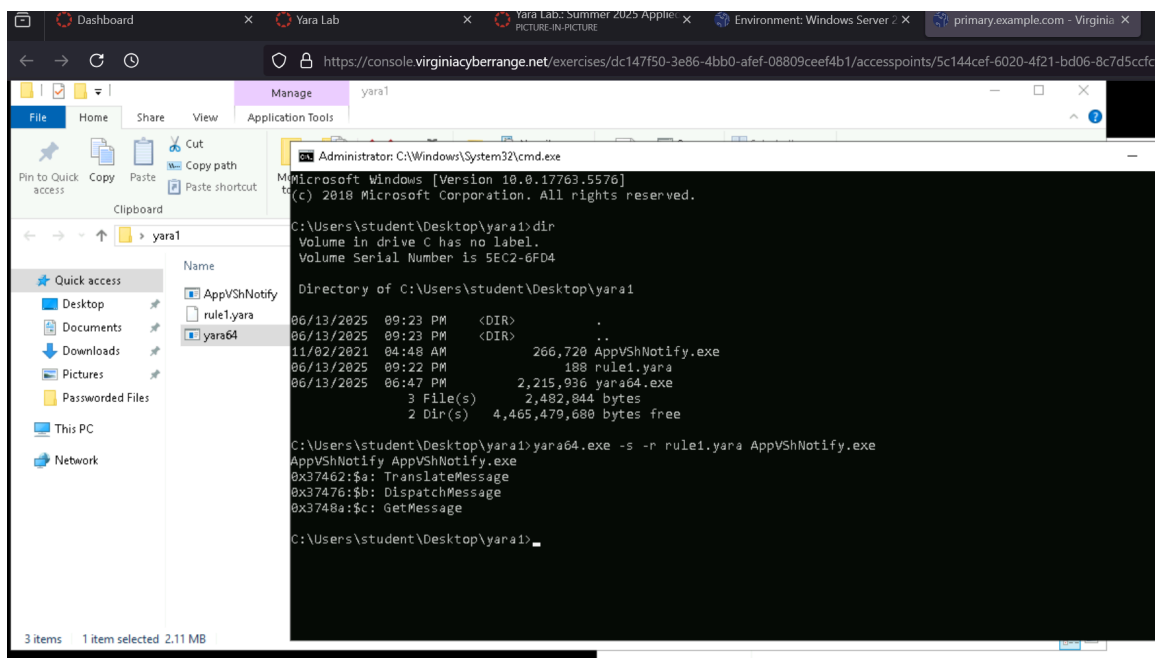
Step 4 – Scan a suspicious Binary

- Make sure the rule file (rule_1.yara) is placed in the same folder as the YARA application as well as the suspicious binary (i.e., AppVShNotify.exe)



Name	Date modified	Type	Size
AppVShNotify	11/2/2021 12:48 AM	Application	261 KB
rule_1.yara	3/8/2023 4:18 PM	YARA File	1 KB
yara64	3/8/2023 2:47 PM	Application	2,164 KB
yara64	3/8/2023 2:47 PM	Application	2,115 KB

- In the command prompt, navigate to the location containing the files.
- Enter the following command `'yara64 -s -r rule_1.yara AppVShNotify.exe'`
- Yara is able to use the rule to scan the file and identify the embedded strings.



```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.17763.5576]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\student\Desktop\yara1>dir
Volume in drive C has no label.
Volume Serial Number is 5EC2-6F04

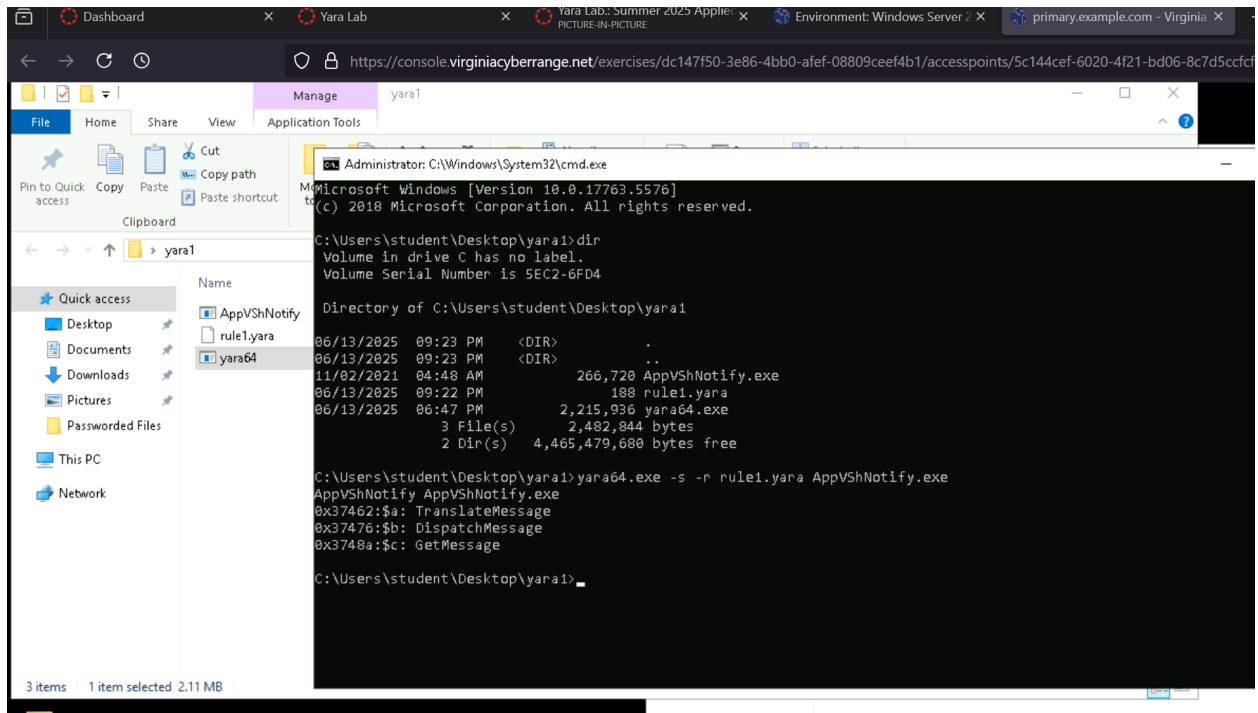
Directory of C:\Users\student\Desktop\yara1

06/13/2025  09:23 PM  <DIR>          .
06/13/2025  09:23 PM  <DIR>          ..
11/02/2021  04:48 AM             266,720 AppVShNotify.exe
06/13/2025  09:22 PM             188 rule1.yara
06/13/2025  09:47 PM      2,215,936 yara64.exe
               3 File(s)      2,482,844 bytes
               2 Dir(s)      4,465,479,680 bytes free

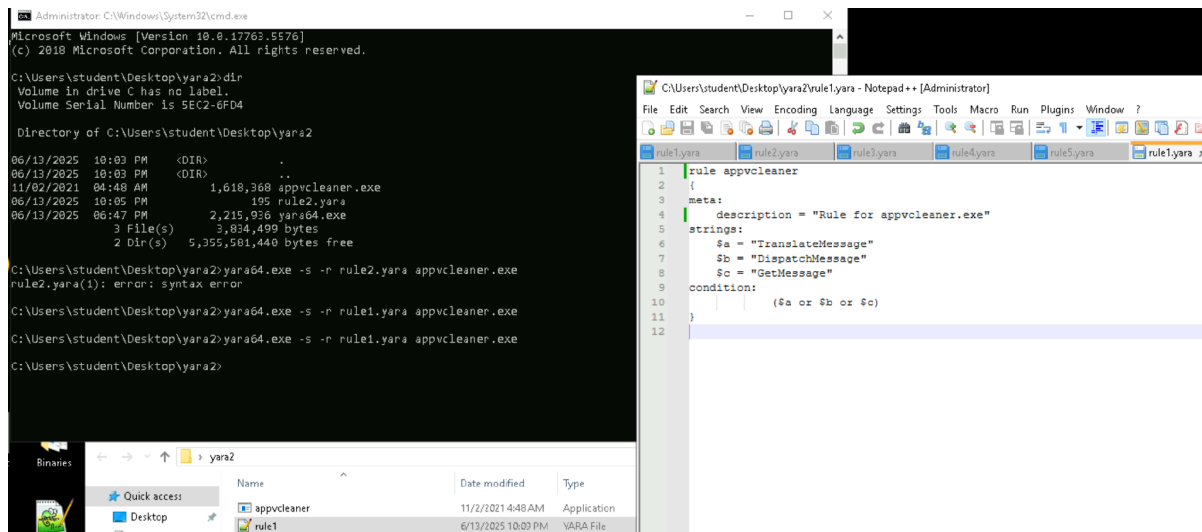
C:\Users\student\Desktop\yara1>yara64.exe -s -r rule1.yara AppVShNotify.exe
AppVShNotify AppVShNotify.exe
0x37462:$a: TranslateMessage
0x37476:$b: DispatchMessage
0x3748a:$c: GetMessage

C:\Users\student\Desktop\yara1>
```

Troy Fracyon
6/15/2025

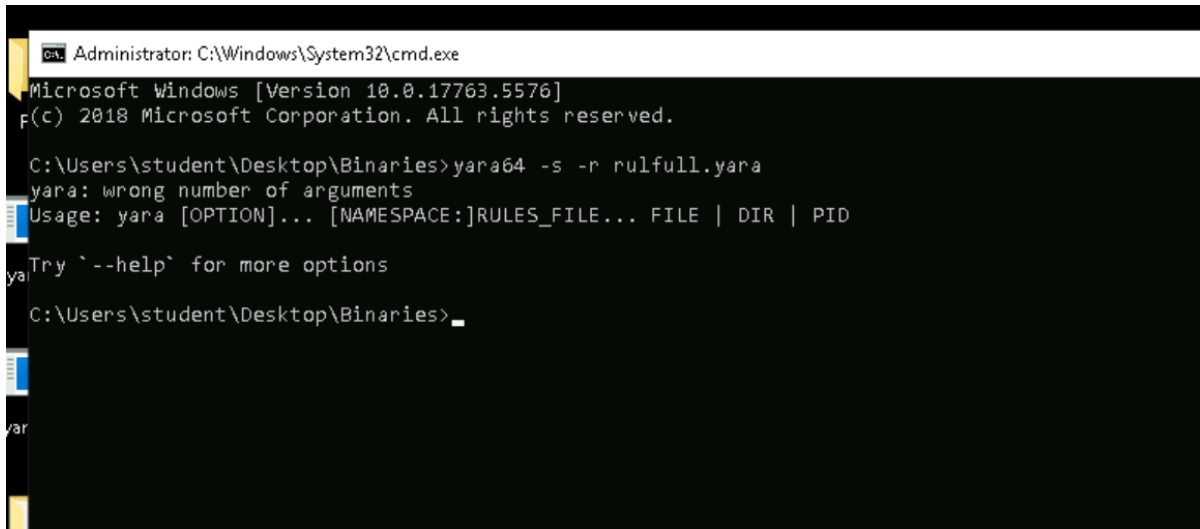


Step 5: Run 'rule_1.yara' against each file in the Binaries folder and report the results.



Check each of the 5 files in the Binaries folder for strings using PeStudio. If you are able to access the strings in a particular binary, create a YARA rule using at least 3 strings. Test your rule and report your results.

1. Place all your rules in one (rules) file and run it against the Binaries folder using the following command: `yara64 -s -r rule_all.yara C:\Users\student\Desktop\Binaries`.



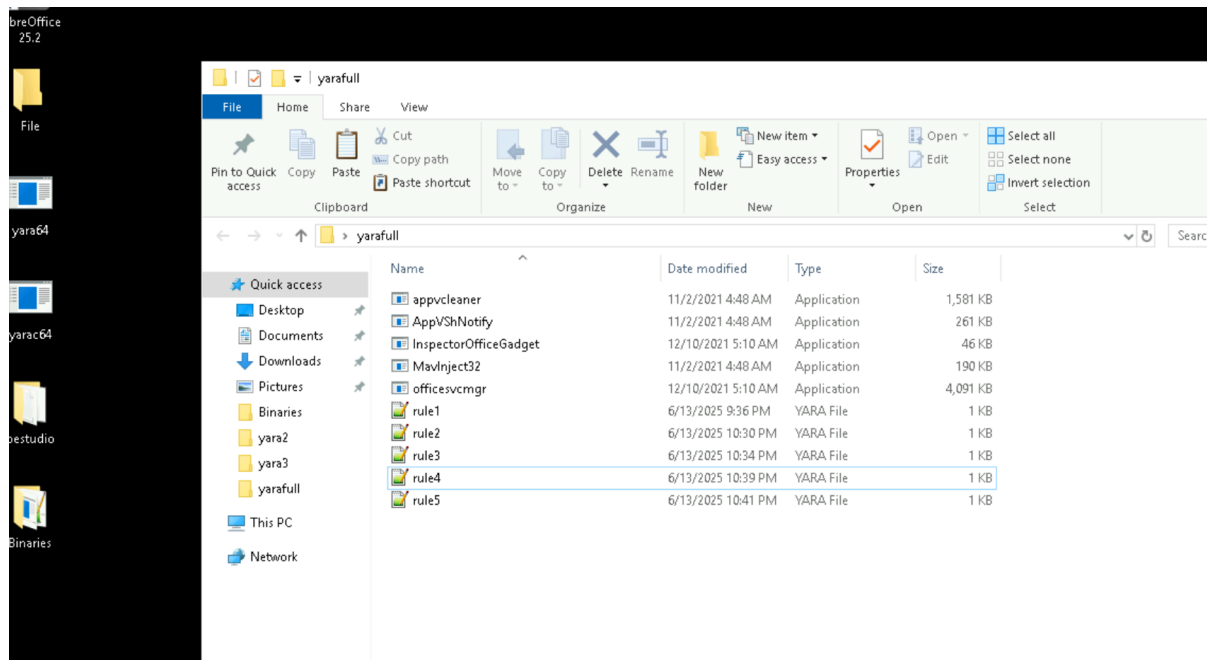
```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.17763.5576]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\student\Desktop\Binaries>yara64 -s -r rulfull.yara
yara: wrong number of arguments
Usage: yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR | PID

Try `--help` for more options

C:\Users\student\Desktop\Binaries>
```

2. Search for an executable file on your computer, create and test YARA rules using at least 5 strings.



Troy Fracyon

6/15/2025

EXAMPLE YARA RULE:

```
rule MyRule
{
  meta:
    description = "Rule for Binaries folder"

  strings:
    // AppvCleaner Strings
    $appvcleaner_1 = "GetCommandLineW"
    $appvcleaner_2 = "CommandLineToArgvW"
    $appvcleaner_3 = "AppvIsvSubsystems64.dll"

    // AppVShNotify Strings
    $shnotify_1 = "TranslateMessage"
    $shnotify_2 = "DispatchMessage"
    $shnotify_3 = "Shell_NotifyIconW"

    // InspectorOfficeGadget Strings
    $inspector_1 = "OfficeClickToRun.exe"
    $inspector_2 = "InitializeSecurityDescriptor"
    $inspector_3 = "StreamingUX"

    // MavInject32 Strings
    $mavinject_1 = "/INJECTRUNNING"
    $mavinject_2 = "CreateRemoteThread"
    $mavinject_3 = "AppvIsvSubsystems32.dll"

    // OfficeSvcMgr Strings
    $officesvc_1 = "StartServiceCtrlDispatcherW"
    $officesvc_2 = "officesvcmgrschedule.xml"
    $officesvc_3 = "OfficeC2RClient.exe"

  condition:
    // Triggers if any of 3 strings for any single file are found
    ($appvcleaner_1 or $appvcleaner_2 or $appvcleaner_3) or
    ($shnotify_1 or $shnotify_2 or $shnotify_3) or
    ($inspector_1 or $inspector_2 or $inspector_3) or
    ($mavinject_1 or $mavinject_2 or $mavinject_3) or
    ($officesvc_1 or $officesvc_2 or $officesvc_3)
}
```

Definition of each file:

Appvleaner – frees up disk space by removing application packages that are no longer in use.

AppVShNotify – Notifies user to updates to virtualized applications

InspectorOfficeGadget – Allows for the streaming of virtualization technology and office applications

MavInject32 – Allows for deployment of virtualized applications

officesvcmgr – manages microsoft office products

References

- How to Install and Run YARA tool on Windows
<https://www.youtube.com/watch?v=rnAEj7cXMTY>
- Malware Analysis Bootcamp - Creating YARA Rules -
<https://www.youtube.com/watch?v=35Exd9GrR5I>
- What are Yara Rules (and How Cybersecurity Analysts Use Them)
https://www.youtube.com/watch?v=BM23_H2GGMA&t
- Finding Evil with YARA - <https://www.youtube.com/watch?v=mQ-mqxOfopk&t>
- Malware Lab - Installing YARA on Linux -
<https://www.youtube.com/watch?v=bQ5TorVUZE4>